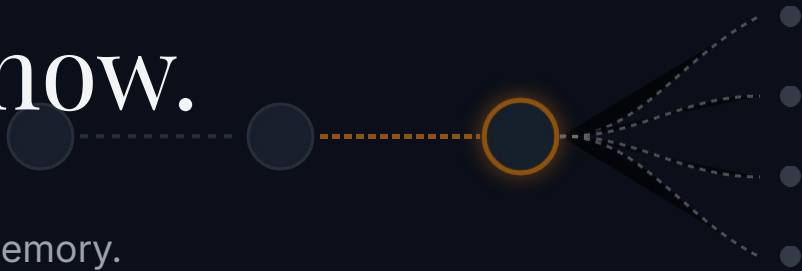


ONEINBOX AI HACKATHON 2026 · FINAL

RAG that knows when it doesn't know.



Self-correcting RAG with a 3-plane context memory.

The generator fires last, not first.

WHAT IT IS

Four honest outcomes

Not a chatbot — an architecture built around one decision: should this question be answered at all?

01

ANSWER

Context is sufficient and consistent.
Generate with chunk-level citations.

02

DUAL-ANSWER

Sources conflict.
Show both values with provenance.
Never arbitrate.

03

CLARIFY

Question is ambiguous. Offer the readings, re-run with the choice.

04

REFUSE

Corpus can't support it. Decline with gap analysis: "covers X, Y; not Z."

WHAT IT INGESTS

Everything, into one graph

DOCUMENTS

PDF · DOCX · PPTX · XLSX · ODT · EPUB · HTML ·
MD · TXT

SCANS & IMAGES

PNG · JPG · TIFF · scanned PDFs with no text
layer

CODE & CHAT

whole repositories, 22+ languages · any chat
transcript



every chunk tagged — file · page · native|ocr · ocr_confidence · content_type

THE PROBLEM

Four ways retrieval quietly lies

BLINDNESS

A scanned page has no text layer. The retriever never sees it, so the fact doesn't exist as far as the system is concerned.

OMISSION

The answer isn't in the corpus at all. Rather than say so, the model composes one from adjacent context.

SILENT ARBITRATION

Two documents disagree. The model picks one, presents it as fact, and never mentions the conflict existed.

AMBIGUITY COLLAPSE

The question had two readings. The model answers one as if the other was never possible.

NOTE ON ARCHITECTURE

These are not prompt problems. Telling a model to "*be careful*" does not fix a missing text layer.

WHY CURRENT SYSTEMS FAIL

Standard RAG has no step that can say no

■ No sufficiency check

Thin context still generates a complete response, even when the required information is entirely missing.

■ No contradiction check

Conflicts between multiple source documents get silently resolved without alerting the end user.

■ No abstention path

A strict "I don't know" or refusal state isn't mathematically reachable in the default generative cycle.

```
query
```

```
↓
```

```
retrieve top-k
```

```
↓
```

```
[ prompt ]
```

```
↓
```

```
generate ✓ always
```

```
# nowhere to refuse
```

WHY IT MATTERS

Three markets this architecture unlocks

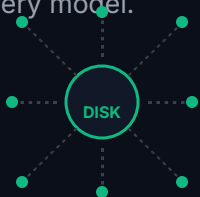
1

TRANSFERABLE CONTEXT ROUTER

Problem: Your context dies with the chat window, locked in one vendor's account.

Ours: Memory is a graph on your disk over MCP as 12 stdio tools. Claude Code to OpenCode seamlessly inherits what was decided.

Value: No vendor lock-in. Portable across every model.



2

ENTERPRISE RAG ECONOMICS

Problem: RAG cost scales with corpus size — re-embedding and full re-indexing on every change.

Ours: Zero LLM call index cost. Built at query time, 2/4-bit quantized, incremental re-parse (~0.1s/edit).

Value: Corpus grows without LLM spend growing with it.



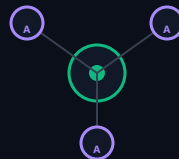
3

SHARED MEMORY FOR AGENTS

Problem: Agents start cold, repeat work, and cannot trace past solutions.

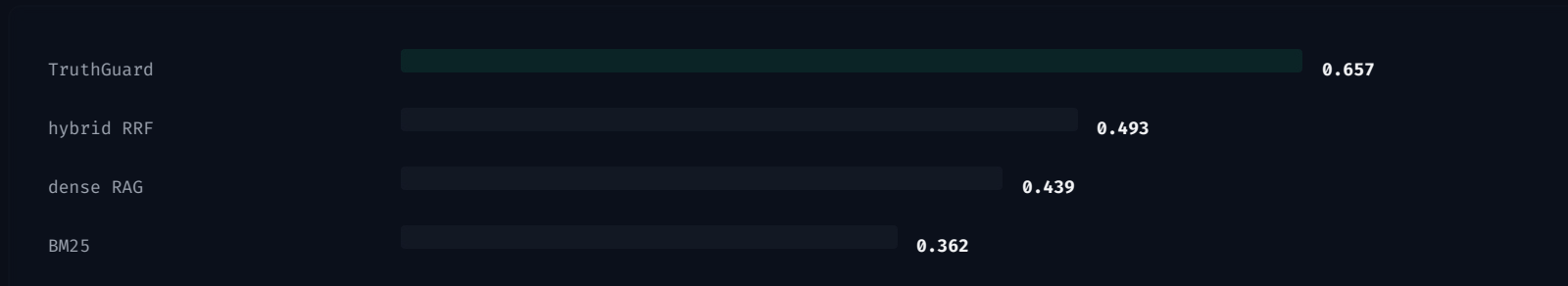
Ours: Agents read/write the same graph. Answers wired as spine nodes to source chunks with replayable traces.

Value: Agents compound knowledge instead of duplicating it.



MEASURED RESULTS

The gate, measured



0.657

LOCOMO recall@10
(n=300) · QA 0.735

0.972

LongMemEval recall@10
(n=470) · QA 0.804

86.7%

correct behavior on
the adversarial
battery

6.7%

hallucination rate
(n=15)

Index build = zero LLM calls. A BM25 baseline inside our own harness scores 0.577 – published so the harness itself can be calibrated.

HOW IT WORKS

The generator fires last, not first

01

INGEST

Layout-aware parsing.
Scan detect: page text
under 50 characters
routes to OCR.
~1000-char chunks split
on headings, never
cutting a table or code
block.

02

RETRIEVE

Three signals in parallel:
dense vectors (MiniLM,
384-dim), BM25
keywords, exact entity
match. RRF fusion →
cross-encoder rerank →
top-50 → top-10.

03

ASSESS ► THE GATE

sufficiency (0 LLM) →
triple extraction (1 LLM)
→ clash check by
dictionary lookup (0
LLM) → answerability
verdict (1 LLM).

04

RESPOND

answer / dual-answer /
clarify / refuse. Bands:
≥0.75 answer, 0.4–0.75
hedge, <0.4 refuse. On
INSUFFICIENT it
rewrites its own query,
max 2 rewrites.

2–4 LLM calls typical, 6 hard cap — and the generator still hasn't run.

Route per page, escalate on evidence

01

TESSERACT

Fast, no weights. Most clean scans stop here.

02

docTR

Detect-then-read. Fixes reading order and poor scans.

03

PADDDLEOCR

Angle-invariant. Rotated and scattered text.

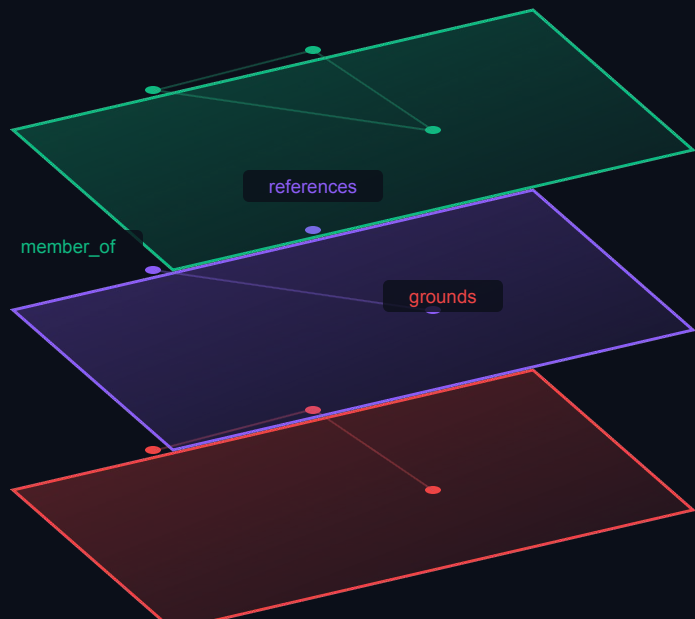
OPT

TIER 2 · VLM

dots.ocr / local vision / Mistral. Off by default. Backend degrades, never fails.

Tesseract 82.1% word recall @ 4.6s/page · docTR 84.4% @ 7.3s · PaddleOCR highest confidence 0.985

One graph for documents, chat, and code



y+ KNOWLEDGE

Provenance chunks, entities, Louvain communities with a compiled truth per topic.

x SPINE

Turns as decision memory: question, answer, confidence, band. Each session its own thread.

y- CODE

Call and import structure plus real function bodies via tree-sitter, across 22+ languages.

"If I change this function, what breaks?"

→ **22 nodes impacted, risk MEDIUM** — 19 code symbols, 2 files, and 1 past chat decision. **0 LLM calls, pure graph traversal.**